MNIST Images Classification MS1 (binary)

Team No: **9**

Contributors:

- **Karim Samer Mostafa** **23p0439**
- **Youssef Maged Morees** **2301081**
- **Omar Gamil Anwar Osman** **23p0438**
- **Paula Hanna Naguib Hanna** **23p0440**

Course: **Introduction to Machine learning** CSE382 (UG2023)

Prof. **Hossam Hassan**

TA. **Eng. Ali Ahmed**

Contents

Project GitHub Repo : https://github.com/karimsamer100/MNIST-Classification-ML-Project 3
1. Introduction 3
2. Problem Definition 3
3. Dataset & Preprocessing 3
4. Feature Representation 4
5. Mathematical Formulation 4
5.1 K-Nearest Neighbors (k = 3) 4
5.2 Logistic Regression (α = 0.1, 1000 iterations) 4
5.3 Gaussian Naive Bayes 5
6. Evaluation Metrics 5
7. Experimental Results 5
7.1 Raw Features 5
7.2 After PCA (50 components) 5
7.3 Key Confusion Matrix Findings (Test Set) 6
8. Model Comparison & Effect of PCA 6
9. Code Structure 7
10. Conclusion 7

Project GitHub Repo : https://github.com/karimsamer100/MNIST-Classification-ML-Project

1. Introduction

This report covers a binary image classification pipeline built from scratch on the MNIST handwritten digit dataset. The task is to distinguish digit 0 from digit 1 using three manually implemented classical ML algorithms: K-Nearest Neighbors, Logistic Regression, and Gaussian Naive Bayes. Each model is evaluated on both raw pixel features and PCA-reduced features. No external machine learning libraries were used. All models, PCA, splitting logic, and metrics were implemented manually using NumPy. Pandas was used only for CSV loading..

2. Problem Definition

Each MNIST image is 28×28 grayscale pixels. After flattening:

$$x \in \mathbb{R}^{784}, \quad y \in \{0, 1\} \quad \text{where } 0 = \text{digit } 0, \ 1 = \text{digit } 1$$

The objective is to learn a function f $\mathbb{R}$^784→ {0,1} that correctly classifies unseen images.

3. Dataset & Preprocessing

The MNIST CSV format provides one row per image: [label, $pixel_1$, …, $pixel_{784}$]. Only classes 0 and 1 were retained. After filtering and splitting:

Split	Shape	Class 0	Class 1
Training	(10133, 784)	4739	5394
Validation	(2532, 784)	1184	1348
Test	(2115, 784)	980	1135

Classes are well-balanced no resampling needed. The preprocessing pipeline:

- Normalize pixels: $x_n = x / 255$ (maps [0,255] → [0,1])
- Stratified 80/20 train/validation split (random state = 42)
- Test set kept separate never touched during development

Additional Notes on Data Distribution

Although the dataset appears balanced at the class level, minor differences exist between the number of samples in each class. However, these differences are not significant enough to require resampling or class balancing techniques.

Moreover, since MNIST images are already standardized to a fixed resolution (28×28), no resizing step was required. This simplifies the preprocessing pipeline and ensures consistency across all samples.

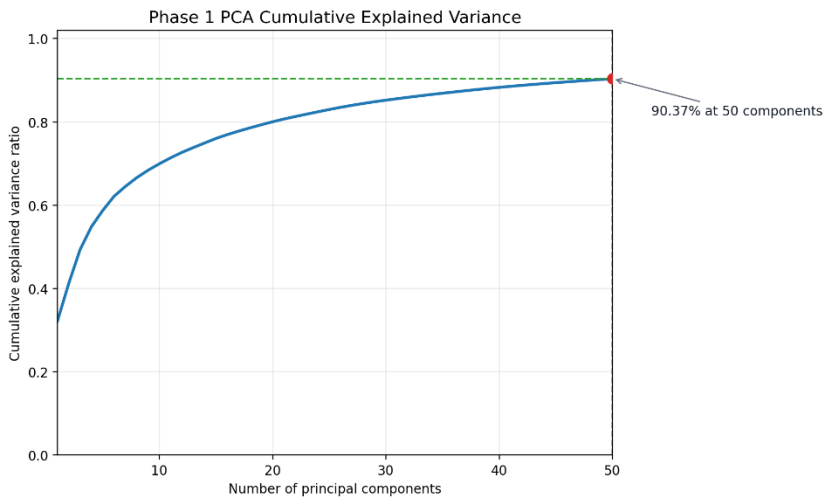
The use of normalization plays a critical role in stabilizing the training process, particularly for Logistic Regression, as it ensures that all features lie within the same numerical range and prevents large gradients during optimization.

4. Feature Representation

Two settings were tested. Raw: all 784 normalized pixels. PCA: top 50 principal components fitted on training data only, then applied to validation and test sets (no data leakage).

Setting	Features	Explained Variance
Raw Pixels	784	
PCA	50	90.37%

PCA reduces dimensionality by 93.6% while retaining 90.37% of total variance.



This confirms that most of the informative variance in the dataset is concentrated in a relatively low dimensional subspace.

5. Mathematical Formulation

5.1 K-Nearest Neighbors (k = 3)

Lazy algorithm no parameter learning. For test sample x, compute Euclidean distance to every training point, find the 3 closest, and return the majority class:

$$d(x, x_i) = \sqrt{\Sigma_j (x_j - x_{ij})^2}$$

No loss function. Prediction cost: O(n × d) per sample.

5.2 Logistic Regression (α = 0.1, 1000 iterations)

Linear score mapped to probability via sigmoid, then thresholded at 0.5:

$$z = w^T x + b \qquad \hat{y} = 1 / (1 + e^{(-z)})$$

Loss Binary Cross-Entropy:

$$J(w,b) = -(1/m)\ \Sigma\ [\ y \log(\hat{y}) + (1-y) \log(1-\hat{y})\ ]$$

Optimization Batch Gradient Descent:

$$w \leftarrow w - \alpha(1/m)X^T(\hat{y}-y) \qquad b \leftarrow b - \alpha(1/m)\Sigma(\hat{y}_i - y_i)$$

5.3 Gaussian Naive Bayes

Applies Bayes' theorem assuming conditional feature independence. For each class c, estimates prior P(c), mean μ^c, and variance σ^{2c} per feature:

```
P(x_j | c) = 1 / sqrt(2πσ_cj²) * exp(-(x_j - μ_cj)² / (2σ_cj²))
```

Prediction: argmax^c [log P(c) + Σ_j log P(x_j | c)] (log-space to avoid underflow)

6. Evaluation Metrics

All metrics implemented manually from the binary confusion matrix (TP, TN, FP, FN):

Metric	Formula
Accuracy	(TP + TN) / Total
Precision	TP / (TP + FP)
Recall	TP / (TP + FN)
F1-Score	2 × (P × R) / (P + R)

7. Experimental Results

7.1 Raw Features

Model	Val Acc	Test Acc	Test Prec	Test Rec	Test F1
KNN (k=3)	0.9992	0.9991	0.9982	1.0000	0.9991
Logistic Regression	0.9972	0.9995	0.9991	1.0000	0.9996
Gaussian Naive Bayes	0.9937	0.9972	0.9974	0.9974	0.9974

The extremely high performance across all models indicates that the binary classification task (digit 0 vs digit 1) is inherently simple and almost perfectly separable in the original feature space.

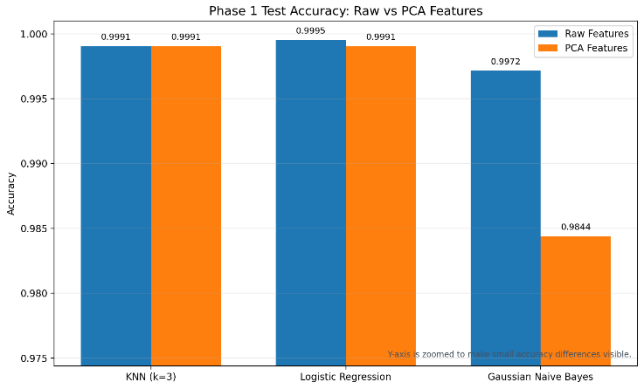
7.2 After PCA (50 components)

Model	Val Acc	Test Acc	Test Prec	Test Rec	Test F1
KNN (k=3)	0.9992	0.9991	0.9982	1.0000	0.9991
Logistic Regression	0.9976	0.9991	0.9982	1.0000	0.9991
Gaussian Naive Bayes	0.9787	0.9844	1.0000	0.9709	0.9852

Impact of Dimensionality Reduction

Reducing the feature space from 784 to 50 dimensions significantly decreases computational complexity while preserving most of the variance in the data. This reduction is particularly beneficial for models such as KNN, where prediction time depends directly on the number of features.

However, dimensionality reduction also introduces a trade-off. While PCA preserves global variance, it may discard some fine-grained pixel-level information that could be useful for classification. This explains the slight performance drop observed in some models after applying PCA.

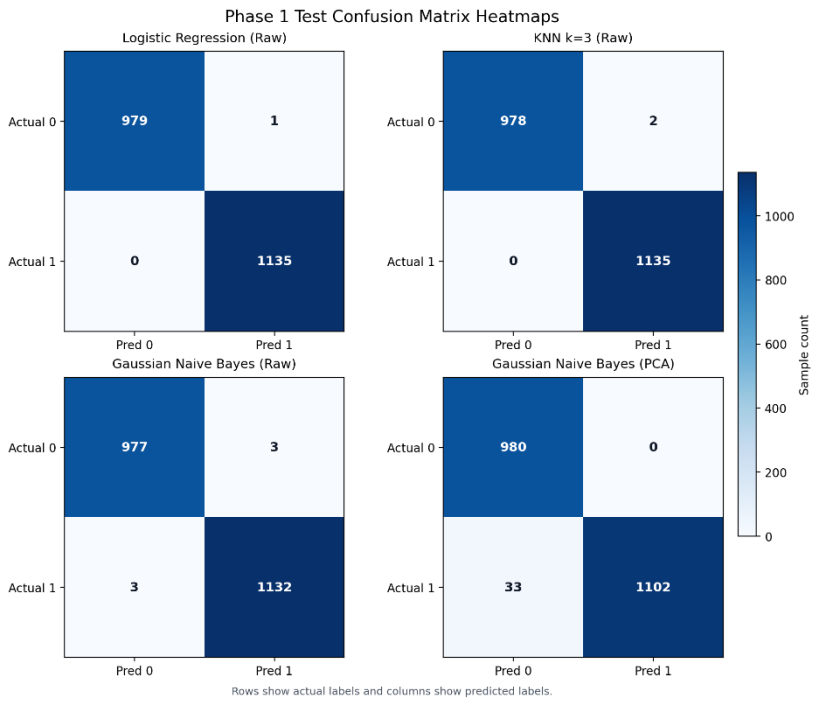


7.3 Key Confusion Matrix Findings (Test Set)

Logistic Regression (Raw) only 1 error total: 1 digit-0 misclassified as digit-1. Best overall.

KNN (Raw) 2 errors: 2 digit-0 images misclassified. Perfect recall on digit-1.

GNB after PCA perfect precision but 33 missed digit-1 samples (recall drops to 0.9709). Model becomes overly conservative when features are PCA-transformed.



8. Model Comparison & Effect of PCA

Model	Raw Test	PCA Test	Change	Key Observation
KNN	0.9991	0.9991	0.0000	PCA preserves neighborhood structure perfectly
Logistic Regression	0.9995	0.9991	−0.0004	Classes nearly linearly separable; PCA negligible
Gaussian NB	0.9972	0.9844	−0.0128	Independence assumption broken by PCA components

KNN is robust to PCA because digit 0 and 1 occupy well-separated regions of the feature space distances are preserved. Logistic Regression confirms the classes are nearly linearly separable in 784D; 50 PCA components retain enough structure for the same conclusion. Gaussian Naive Bayes performance degrades after PCA because the transformed features no longer follow simple per-feature Gaussian distributions, and some class-discriminative information may be compressed or mixed during the transformation.

9. Code Structure

Module	Responsibility
data_module.py	Load CSV, filter classes, normalize, stratified split
features_module.py	PCA from scratch: fit, transform, fit_transform
models_module.py	KNN, Logistic Regression, Gaussian NB all from scratch
evaluation_module.py	Accuracy, precision, recall, F1, confusion matrix
main.py	Full Phase 1 pipeline orchestration

10. Conclusion

All Phase 1 requirements were met: mathematical formulation, preprocessing, three from-scratch model implementations, PCA evaluation, and full metric reporting. The best result was Logistic Regression on raw features accuracy 0.9995, F1 0.9996 confirming that digit 0 vs 1 is nearly linearly separable. KNN matched it after PCA. Gaussian Naive Bayes performed well on raw features but degraded under PCA due to its independence assumption. Classical ML algorithms with proper preprocessing are sufficient for near-perfect accuracy on this binary task.

These results demonstrate that even simple linear models can achieve near-optimal performance when the data is well-structured and properly preprocessed.

Limitations:

Despite the excellent performance achieved, this phase focuses on a simplified binary task. The results do not necessarily generalize to more complex multi-class classification problems, where feature relationships become more intricate and model limitations become more apparent.